Figure 7.15. Adding estimate health to our other metrics for predictability

Figure 7.16. A representation of the release process before the team moved to CD

Figure 7.17. A representation of the new process they used to deploy code continuously

Figure 7.18. Part 1 of the mind-mapping exercise

Figure 7.19. Getting to the individual data points for this metric

Figure 7.20. Using Dashing to create an easy-to-read dashboard for the larger team

Figure 7.21. A second team decided to use the same rating system but tweak the composition of the metric to fit their process.

Chapter 8. Measuring the technical quality of your software

Figure 8.1. The code “ilities” illustrated through the life of a software product

Figure 8.2. The code “ilities” in relation to the components of the build lifecycle and the tools used to measure them

Figure 8.3. Grouping systems used in the development lifecycle together by the two parent code “ilities”

Figure 8.4. Components of lead time and MTTR as they move through the delivery process

Figure 8.5. A mind map breakdown of what you need to measure maintainability

Figure 8.6. An example breakdown of MTTR over four releases; testing takes much more time than anything else in the fix cycle.

Figure 8.7. Another example breaking down MTTR over four releases: with testing fully automated, MTTR goes way down. The next focus for improvement should be the build process.

Figure 8.8. The amount of code that changes for all hot fixes in a month

Figure 8.9. An example breakdown of code coverage in SonarQube